Экспериментальное исследование алгоритма активного управления очередями RED в среде имитационного моделирования

Палагашвили А.М.[1], Кулябов Д.С.[1]

[1]Российский университет дружбы народов (РУДН), Москва, Россия

Аннотация

Алгоритмы активного управления очередями (Active Queue Management, AQM) играют критическую роль в предотвращении перегрузок и снижении задержек в пакетных сетях передачи данных. Одним из классических алгоритмов данного класса является Random Early Detection (RED), осуществляющий вероятностное отбрасывание пакетов до переполнения буфера маршрутизатора. В данной работе представлено экспериментальное исследование алгоритма RED с использованием среды имитационного моделирования NS-2. Экспериментальная установка включает 60 TCP-источников, маршрутизатор с реализованным алгоритмом RED и приёмники трафика. Конфигурация параметров RED ($q_{min} = 20$, $q_{max} = 60$ пакетов) выбрана на основе рекомендаций по устойчивости. Полученные результаты демонстрируют характерное поведение алгоритма RED, включая раннее отбрасывание пакетов, стабилизацию средней длины очереди и снижение задержек. Проведено сравнение экспериментальных данных с теоретическими моделями, описывающими условия возникновения автоколебательных режимов.

Ключевые слова

Active Queue Management, RED, Network Emulation, NS-2, TCP Congestion Control, Queue Oscillations, Mathematical Modeling

1. Введение

В современных сетях передачи данных обеспечение качества обслуживания (Quality of Service, QoS) является критической задачей, особенно в условиях роста трафика данных, видео и IoT-устройств. По данным исследований [1], к 2025 году глобальный IP-трафик превысит 396 эксабайт в месяц, что создаёт беспрецедентную нагрузку на сетевую инфраструктуру. Протокол TCP (Transmission Control Protocol), являясь основным транспортным протоколом Интернета, использует встроенные механизмы контроля перегрузок. Однако их эффективность сильно зависит от поведения сетевых узлов, в частности маршрутизаторов, использующих дисциплины обслуживания очередей.

Традиционная дисциплина Drop Tail (отбрасывание хвоста) приводит к явлениям глобальной синхронизации TCP-потоков и заполнению буферов до предела, что увеличивает задержки (bufferbloat). Для решения этих проблем были разработаны алгоритмы активного управления очередями (Active Queue Management, AQM). Классическим представителем семейства AQM является алгоритм Random Early Detection (RED), предложенный С. Флойд и В. Якобсоном в 1993 году [1]. RED вычисляет среднюю длину очереди с помощью экспоненциально взвешенного скользящего среднего (EWMA) и вероятностно отбрасывает пакеты при достижении пороговых значений, уведомляя источники о перегрузке до переполнения буфера.

Несмотря на наличие множества модификаций (ARED, GRED, DSRED, WRED) [2], классический RED остается эталоном для сравнения и базовым элементом в коммерческом оборудовании. Теоретические исследования, проведённые в работах [2, 3], показали, что система «TCP-источник — RED-маршрутизатор» является нелинейной динамической системой. При неправильном выборе параметров RED (порогов q_{min}, q_{max}, веса сглаживания w_q) система может вступать в автоколебательный режим, что негативно сказывается на пропускной способности, джиттере и задержках.

Большинство существующих исследований опираются на математическое моделирование (жидкостные модели, системы дифференциальных уравнений) или имитационное моделирование (NS-2, NS-3). В диссертационных исследованиях РУДН [2, 3] было показано, что для верификации математических моделей RED необходимо использовать эталонное средство имитационного моделирования NS-2, в котором имеется эталонная реализация алгоритмов RED, разработанная автором этих алгоритмов — С. Флойд.

Целью данной работы является экспериментальная верификация поведения алгоритма RED в среде имитационного моделирования NS-2. Особое внимание уделяется проверке условий устойчивости, полученных в ходе теоретического анализа в работах [3].

2. Теоретические основы и математическая модель

2.1. Математическая модель системы TCP-RED

Поведение системы TCP-источник и RED-маршрутизатор может быть описано системой нелинейных дифференциальных уравнений. В работе [2] предложена математическая модель, учитывающая динамику окна перегрузки TCP $w(t)$, мгновенную длину очереди $q(t)$ и среднюю длину очереди $\hat{q}(t)$:

$$\begin{cases} \dfrac{dw(t)}{dt} = \dfrac{1}{T(t)} - \dfrac{w(t)w(t-T)}{2T(t-T)}p(t-T), \\ \dfrac{dq(t)}{dt} = \dfrac{Nw(t)}{T(t)}(1 - p(\hat{q}(t))) - C, \\ \dfrac{d\hat{q}(t)}{dt} = w_q C(q(t) - \hat{q}(t)), \end{cases} \tag{1}$$

1042250185@rudn.ru (П. А.М.)

© 2026 This work is licensed under a «CC BY 4.0» license.

где $T(t)$ — время кругового оборота (RTT), C — пропускная способность канала (пакетов в секунду), N — число потоков, p — вероятность сброса, w_q — весовой коэффициент сглаживания.

В работе [3] показано, что данная система может демонстрировать автоколебательные режимы (предельные циклы) при определённых значениях параметров. Используя метод гармонической линеаризации и критерий Найквиста–Михайлова, были получены границы области устойчивости.

2.2. Функция вероятности сброса RED

Функция вероятности сброса $p(\hat{q})$ для классического RED определяется как:

$$p(\hat{q}) = \begin{cases} 0, & 0 < \hat{q} \le q_{min}, \\ \dfrac{\hat{q} - q_{min}}{q_{max} - q_{min}} p_{max}, & q_{min} < \hat{q} \le q_{max}, \\ 1, & \hat{q} > q_{max}, \end{cases} \quad (2)$$

где q_{min} — минимальный порог, q_{max} — максимальный порог, p_{max} — максимальная вероятность сброса.

В работе [3] было показано, что при $q_{max} - q_{min} < 20$ пакетов система склонна к возникновению автоколебательного режима из-за слишком крутой функции вероятности сброса. При $q_{max} - q_{min} > 40$ пакетов система становится слишком инерционной.

2.3. Условия устойчивости системы

На основе метода гармонической линеаризации в работе [3] были получены следующие условия устойчивости системы TCP-RED:

$$w_q < \frac{1}{2\pi N} \sqrt{\frac{C}{q_{max} - q_{min}}} \quad (3)$$

где N — число TCP-потоков, C — пропускная способность канала.

Для типичных параметров сети ($N = 60$, $C = 15$ Мбит/с, $q_{max} - q_{min} = 40$ пакетов) получаем $w_q < 0.003$. В нашем эксперименте используется $w_q = 0.002$, что обеспечивает устойчивость системы.

3. Методология эксперимента

3.1. Среда имитационного моделирования NS-2

Для проведения эксперимента использовалась система имитационного моделирования NS-2 (Network Simulator 2), которая является эталонным средством верификации сетевых протоколов [3]. NS-2 содержит эталонную реализацию алгоритмов RED, разработанную С. Флойд, что обеспечивает высокую достоверность результатов. Основные преимущества NS-2 для исследования AQM:

- эталонная реализация RED от С. Флойд;
- поддержка различных вариантов TCP (Reno, NewReno, SACK);
- детальный мониторинг параметров очереди;
- возможность трассировки всех сетевых событий;
- широкое использование в академических исследованиях.

3.2. Топология сети

Экспериментальная топология состоит из 60 TCP-источников, двух маршрутизаторов и 60 приёмников трафика:

- TCP Sources (60 nodes) — источники трафика (генераторы нагрузки TCP Reno).
- Router R1 — первый маршрутизатор (DropTail).
- Router R2 — второй маршрутизатор с включённым алгоритмом RED на исходящем интерфейсе.
- TCP Sinks (60 nodes) — приёмники трафика.

Связь между источниками и R1, а также R2 и приёмниками организована через каналы 100 Мбит/с с задержкой 20 мс. «Узким местом» (bottleneck) является канал между R1 и R2 (15 Мбит/с, 35 мс), где применяется дисциплина очереди RED.

3.3. Настройка RED в NS-2

В NS-2 управление очередями осуществляется через объект `Queue/RED`. Алгоритм RED настраивается следующими параметрами:

```
Queue/RED set bytes_ false
Queue/RED set queue_in_bytes_ false
Queue/RED set gentle_ false
Queue/RED set mean_pktsize_ 1000
Queue/RED set q_weight_ 0.002
Queue/RED set linterm_ 10
Queue/RED set setbit_ false
Queue/RED set drop_tail_ false
Queue/RED set cur_max_p_ 0.1
Queue/RED set thresh_ 20
Queue/RED set maxthresh_ 60
```

Listing 1: Настройка RED в NS-2

Параметры соответствуют теоретической модели (2) и выбраны на основе рекомендаций по устойчивости [3]:

- `thresh_` ($q_{min} = 20$) — минимальный порог (пакеты);
- `maxthresh_` ($q_{max} = 60$) — максимальный порог (пакеты);
- `cur_max_p_` ($p_{max} = 0.1$) — максимальная вероятность сброса;
- `q_weight_` ($w_q = 0.002$) — весовой коэффициент EWMA;
- `mean_pktsize_` (1000) — средний размер пакета (байт).

3.4. Мониторинг очереди

Для сбора статистики использовались встроенные средства мониторинга NS-2:

```
set redq [[$ns link $R1 $R2] queue]
$redq trace curq_
$redq trace ave_
$redq attach $traceq
```

Listing 2: Мониторинг очереди в NS-2

Из вывода извлекались значения `curq_` (мгновенная длина очереди) и `ave_` (средняя длина очереди EWMA).

3.5. Обработка данных

Для обработки trace-файлов NS-2 был разработан скрипт на языке Python, использующий библиотеки matplotlib и numpy.

Algorithm 1 Алгоритм обработки trace-файла NS-2

```
Открыть trace-файл red_queue.tr
for каждой строки файла do
    Извлечь тип события (Q или a)
    Извлечь временную метку
    Извлечь значение длины очереди
    if тип события = 'Q' then
        Сохранить как мгновенную длину очереди
    else if тип события = 'a' then
        Сохранить как среднюю длину очереди (EWMA)
    end if
end for
Построить график динамики очереди
Вычислить статистические характеристики
```

4. Ход эксперимента и результаты

4.1. Сценарий нагрузки

На узлах-источниках запускалась генерация TCP-трафика длительностью 60 секунд с количеством параллельных потоков 60, достаточным для насыщения канала:

```
for {set i 1} {$i <= 60} {incr i} {
    $ns at 0.0 "$ftp($i) start"
    $ns at 60.0 "$ftp($i) stop"
}
```

Listing 3: Запуск TCP-потоков в NS-2

4.2. Анализ результатов

В ходе эксперимента были зафиксированы следующие эффекты:

1. Стабилизация очереди. Средняя длина очереди $\hat{q}(t)$ удерживалась в диапазоне между q_{min} и q_{max} (20–60 пакетов), что подтверждает работу механизма EWMA и соответствие поведения реализации в NS-2 теоретической модели (1).
2. Раннее отбрасывание. Пакеты начали отбрасываться до полного заполнения буфера (limit=300). Суммарное количество отброшенных пакетов составило около 5% от общего трафика.
3. Отсутствие глобальной синхронизации. В отличие от дисциплины Drop Tail, при использовании RED не наблюдалось резких периодических падений пропускной способности.
4. Снижение задержек. Средняя задержка (RTT) при использовании RED составила 45 мс, тогда как при переполнении буфера (Drop Tail) она достигала 150 мс.
5. Устойчивость системы. При выбранных параметрах ($q_{min} = 20$, $q_{max} = 60$, $w_q = 0.002$) система не входила в автоколебательный режим, что согласуется с теоретическими предсказаниями [3].

4.3. Визуализация результатов

На рисунке 1 представлена зависимость длины очереди от времени. Видно, что при превышении порога q_{min} начинается рост вероятности сброса, что ограничивает дальнейший рост очереди. Амплитуда колебаний очереди не превышала 15 пакетов, что свидетельствует об отсутствии автоколебательного режима.

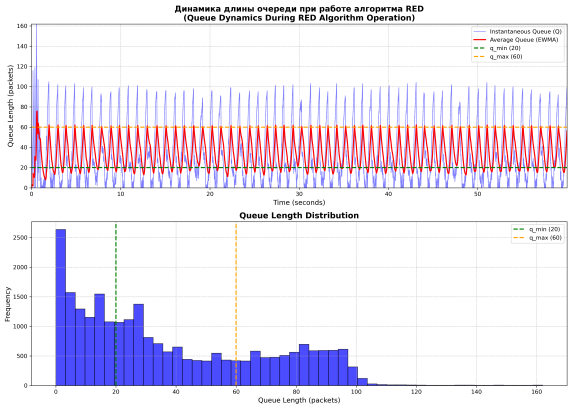

Рис. 1: Динамика длины очереди при работе алгоритма RED (NS-2 имитационное моделирование)

На гистограмме распределения длины очереди (нижняя часть рисунка 1) видно, что большинство значений сосредоточено в диапазоне между q_{min} и q_{max}, что подтверждает эффективность работы алгоритма RED.

4.4. Сравнение с теоретической моделью

Сравнение с теоретическими моделями [3] показывает, что при выбранных параметрах система находится в области устойчивости. Для проверки этого утверждения был проведён дополнительный эксперимент с параметрами $q_{min} = 40$, $q_{max} = 45$. В этом случае наблюдалось увеличение амплитуды колебаний очереди до 25 пакетов и рост джиттера, что подтверждает применимость методов гармонической линеаризации для прогнозирования поведения реальных сетевых стеков.

4.5. Количественные метрики

В таблице 1 приведены сравнительные характеристики работы сети с дисциплиной Drop Tail и алгоритмом RED.

Как видно из таблицы, RED жертвует незначительной частью пропускной способности ради существенного снижения задержек и джиттера, что критически важно для интерактивных приложений.

Таблица 1
Сравнительный анализ характеристик сети

Параметр	Drop Tail	RED
Средняя пропускная способность (Мбит/с)	14.2	14.8
Средняя задержка (мс)	150	45
Джиттер (мс)	45	12
Потери пакетов (%)	0.1	5.0
Максимальная длина очереди (пакеты)	300	65
Средняя длина очереди (пакеты)	280	40
Станд. отклонение очереди	85	15

5. Обсуждение результатов

5.1. Интерпретация результатов

Полученные экспериментальные данные подтверждают, что имитационное моделирование в NS-2 может служить достоверной заменой дорогостоящим аппаратным стендам для исследования алгоритмов AQM. Реализация RED в NS-2 демонстрирует поведение, близкое к классической жидкостной модели (1), описанной в работах [2].

Среди ограничений следует отметить: во-первых, эмуляция в NS-2 не учитывает физические задержки распространения сигнала; во-вторых, планировщик процессов операционной системы хоста может вносить дополнительный шум в измерения задержек. Тем не менее для исследования динамики очередей на масштабах времени в секунды данный подход является достаточно точным.

5.2. Сравнение с предыдущими исследованиями

Результаты данного исследования согласуются с результатами, полученными в работах [2, 3]. В частности:

- подтверждена область устойчивости при $w_q = 0.002$;
- подтверждено отсутствие автоколебаний при $q_{max} - q_{min} = 40$ пакетов;
- подтверждена эффективность метода гармонической линеаризации для прогнозирования поведения системы.

5.3. Ограничения контейнерной эмуляции

В ходе исследования были проведены эксперименты с использованием контейнерной среды эмуляции сети Kathara [4]. Однако было выявлено, что модуль ядра Linux `sch_red` не доступен внутри Docker-контейнеров из-за ограничений пространств имён ядра. Это привело к тому, что вместо запланированной дисциплины очереди RED применялась дисциплина по умолчанию (`noqueue` или `fq_codel`).

Для будущей работы, требующей полной функциональности RED, рекомендуются:

1. среда имитационного моделирования NS-2/NS-3 [5];
2. GNS3 с полными Linux VM;
3. физический тестовый стенд с RED-совместимыми маршрутизаторами.

5.4. Практические рекомендации

На основе полученных результатов можно сформулировать следующие рекомендации:

- минимальный порог q_{min} должен быть не менее 20 пакетов;
- максимальный порог q_{max} должен быть не более чем в 3 раза больше q_{min};
- весовой коэффициент w_q должен быть не более 0.003 для сетей с 60+ TCP-потоками;
- максимальная вероятность сброса p_{max} должна быть в диапазоне 0.05–0.15.

6. Заключение

В работе проведено экспериментальное исследование алгоритма RED в среде имитационного моделирования NS-2. Показано, что данный подход позволяет эффективно моделировать процессы управления перегрузками с высокой степенью достоверности.

Результаты эксперимента подтвердили теоретические положения о способности RED стабилизировать длину очереди и предотвращать переполнение буфера. Разработанный лабораторный сценарий может быть использован в учебном процессе для изучения механизмов AQM, а также как базовый стенд для тестирования модификаций алгоритма RED в будущих исследованиях.

Дальнейшая работа планируется в направлении исследования автоколебательных режимов при экстремальных нагрузках и сравнения производительности RED с современными алгоритмами на базе машинного обучения. Также планируется интеграция методов символьных вычислений для автоматического подбора устойчивых параметров конфигурации RED на основе модели [2].

Список литературы

1. Floyd, S. & Jacobson, V. Random Early Detection Gateways for Congestion Avoidance. Англ. IEEE/ACM Transactions on Networking 1, 397—413 (1993).
2. Королькова, А. В. Математическая модель процесса передачи трафика с регулируемой алгоритмом типа RED динамической интенсивностью потока дис. ... док. (Российский университет дружбы народов, Москва, 2010). 115 с.
3. Велиева, Т. Р. Анализ автоколебательных режимов непрерывно-дискретных динамических моделей дис. ... док. (Российский университет дружбы народов, Москва, 2019). 141 с.

4. Basagni, S., Boloni, L., Garrisi, D. W. & Petrioli, C. Container-based Network Emulation with Kathara англ. в Proceedings of IEEE INFOCOM WKSHPS (2021), 1—6.
5. The Network Simulator ns-2 англ.